

API и машинное обучение

API и Python

Когда мы хотим «поговорить» с моделью, получить данные или запустить агент, чаще всего взаимодействие происходит через **API** — программный интерфейс, который позволяет одной программе обращаться к другой. В контексте современных LLM и мультиагентных систем, почти всё начинается с **REST API**, построенного на **HTTP-протоколе**.

 **API (Application Programming Interface)** — это интерфейс, через который одна программа может взаимодействовать с другой. Это как меню в ресторане: вы не идёте на кухню, а просто говорите официанту, что хотите.

Пример:

- Вы отправляете запрос к API OpenAI: "Хочу ответ от GPT-4 по вот такому промπτ" .
- Сервер обрабатывает запрос и отправляет ответ: "Вот твой результат" .

 **HTTP (HyperText Transfer Protocol)** — это протокол, по которому ведётся общение в интернете. Он описывает, как клиент (например, Python-программа) и сервер (например, OpenAI API) обмениваются данными.

HTTP-запрос состоит из:

- **метода** (GET, POST и др.),
- **URL-адреса (endpoint)**,
- **заголовков (headers)**,
- (иногда) **тела запроса (payload/body)**.

 **REST (Representational State Transfer)** — это архитектурный стиль, который использует HTTP для работы с ресурсами. REST API — это API, которое следует этим принципам.

REST опирается на простые правила:

- Использовать стандартные HTTP-методы: GET , POST , PUT , DELETE
- Идентифицировать ресурсы по URL (endpoint)
- Работать по принципу «запрос → ответ»

В таблице ниже вы можете видеть более подробное пояснение основных HTTP-методов:

Метод	Назначение	Пример
GET	Получить данные	Получить информацию о пользователе
POST	Отправить/создать данные	Отправить prompt в LLM
PUT	Обновить данные	Обновить запись в базе
DELETE	Удалить данные	Удалить объект по ID

Теперь давайте обсудим ключевые термины и понятия, которые необходимо знать.

Endpoint - это URL, по которому можно достучаться до ресурса на сервере.

Пример:

▼

```
1 https://api.openai.com/v1/chat/completions
```

Payload (или body) - это тело запроса. Тело запроса — данные, которые мы отправляем, особенно в **POST** и **PUT**.

Пример:

▼

```
1 {
2   "model": "gpt-4",
3   "messages": [{"role": "user", "content": "Hello, GPT!"}]
4 }
```

Headers (заголовки) — метаданные запроса. Например, кто отправляет, в каком формате, какой токен авторизации.

Пример:

▼

```
1 Content-Type: application/json
2 Authorization: Bearer YOUR_API_KEY
```

HTTP-ответ всегда приходит с **кодом статуса** — числом, которое указывает, как завершился запрос. Вот примеры таких статусов:

Код	Значение	Пример
200	OK	Всё хорошо
201	Created	Ресурс создан
400	Bad Request	Ошибка в запросе
401	Unauthorized	Неверный ключ
404	Not Found	Endpoint не найден
500	Internal Server Error	Ошибка на стороне сервера

🎓 Резюме

- **API** — интерфейс между программами.
- **REST** — способ общения по HTTP, основанный на методах **GET**, **POST** и др.
- **HTTP-запрос** состоит из метода, URL (endpoint), заголовков (headers) и, при необходимости, тела (payload).
- Ответ содержит **код статуса**, который всегда надо проверять.

Работа с `requests`: отправка параметров, тела, заголовков и обработка JSON-ответов

Библиотека `requests` — это **стандарт де-факто** для отправки HTTP-запросов в Python. Она проста, читаема и активно используется для работы с REST API, в том числе с LLM, векторными базами и агентными системами.

Если библиотека не установлена, то ее надо установить:

```
1 pip install requests
```

Импорт в Python-скрипте:

```
1 import requests
```

Часто при `GET` запросе необходимо передать параметры (например, фильтрацию или ключевые слова). Они передаются через **query string** (`?key=value`).

Пример:

```
1 import requests # Импортируем библиотеку requests для отправки HTTP-запросов
2
3 url = "https://httpbin.org/get" # Задаём URL-адрес для GET-запроса
4 params = {"search": "langchain", "limit": 5} # Параметры запроса в виде словаря
5
6 response = requests.get(url, params=params) # Отправляем GET-запрос с указанным URL и параметрами
7
8 print("URL:", response.url) # Полный URL с параметрами
9 print("Ответ:", response.json()) # Выводим ответ сервера, преобразованный из формата JSON в словарь Python
```

Метод `POST` используется для отправки данных. Как правило, API принимает данные в формате **JSON**, который передаётся в теле запроса (payload).

Пример:

```
1 import requests # Импортируем библиотеку requests для выполнения HTTP-запросов
2
3 url = "https://httpbin.org/post" # Указываем URL-адрес, на который будет отправлен POST-запрос
4 payload = {"prompt": "Привет, как дела?", "model": "gpt-4"} # Формируем тело запроса — словарь с данными,
    # которые будут отправлены в формате JSON
5
6 response = requests.post(url, json=payload) # Отправляем POST-запрос по заданному URL с JSON-данными
7
8 print("Статус:", response.status_code) # Выводим HTTP-статус код ответа (например, 200 означает успех)
9 print("Ответ JSON:", response.json()) # Выводим тело ответа, преобразованное из JSON в словарь Python
```

Параметр `json=...` автоматически сериализует данные в JSON и добавляет заголовок `Content-Type: application/json`.

Заголовки (`headers`) нужны для:

- Указания формата (`Content-Type` , `Accept`)
- Авторизации (`Authorization`)
- Управления версионированием

Пример с токеном:

```
1
2 url = "https://api.example.com/data" # Задаём URL-адрес API, к которому будет отправлен GET-запрос
3 headers = {
4     "Authorization": "Bearer YOUR_API_KEY",
5     "Accept": "application/json"
6 } # Заголовки запроса:
7 # Authorization — используется для передачи API-ключа (токена) доступа
8 # Accept — указывает, что клиент ожидает ответ в формате JSON
9
10 response = requests.get(url, headers=headers) # Отправляем GET-запрос с указанными заголовками
11
12 print("Ответ:", response.status_code) # Выводим статус-код ответа (например, 200 — успешно, 401 — ошибка
    авторизации и т.д.)
```

Чаще всего API возвращает ответ в формате JSON. С ним удобно работать как со словарём:

```
1 data = response.json()
2 print(data["key"])
```

Если API вернул не-JSON (например, HTML), метод `.json()` выбросит исключение:

```
1 try:
2     data = response.json()
3 except ValueError:
4     print("Ошибка: ответ не в формате JSON")
```

Важно всегда проверять:

- Успешен ли запрос (`response.ok` , `response.status_code`)
- Корректен ли формат ответа

Пример с проверкой ошибок:

▼

```
1 response = requests.get("https://api.github.com")
2
3 if response.ok:
4     print("Успешно:", response.status_code)
5     print(response.json())
6 else:
7     print("Ошибка:", response.status_code)
```

Параметры requests

Параметр	Описание	Используется в
params	Параметры URL (query string)	GET
json	JSON-представление тела запроса	POST, PUT
headers	Заголовки HTTP-запроса	Все методы
data	Тело запроса в форме ключ-значение	POST, PUT

🎓 Резюме

- requests — мощный и удобный инструмент для общения с REST API.
- Передавать параметры можно через params , тело — через json , заголовки — через headers .
- Ответ API обрабатывается методом .json() , если ответ в правильном формате.
- Никогда не игнорируйте status_code — это ваш первый индикатор успеха или ошибки.

Авторизация — API-ключи, токены, OAuth2

Большинство современных API требуют **авторизации** — подтверждения, что у вас есть право на доступ к ресурсу. Давайте разберём, какие бывают механизмы авторизации, как использовать **токены**, **API-ключи** и **OAuth2**.

Авторизация — процесс подтверждения ваших прав на выполнение запроса. Даже если endpoint публичный, для многих операций (например, отправки запроса в LLM или доступа к облачному хранилищу) нужно предъявить **токен** или **ключ**.

Примеры:

- OpenAI требует **API-ключ** в заголовке `Authorization`
- GitHub API — **токен доступа**
- Hugging Face — **токен авторизации**

API-ключи и токены: различия и сходства

Параметр	API-ключ	Токен доступа (access token)
Простота	Статичен, один на проект	Может быть временным
Безопасность	Хранится как пароль	Может иметь ограниченные права
Использование	В заголовке запроса	В заголовке или URL
Источник	Получаете в интерфейсе	Получаете через процесс авторизации

Пример использования API-ключа

Предположим, у вас есть ключ к OpenAI API: `sk-abc123...`

```
1 # Импортируем библиотеку requests для работы с HTTP-запросами
2 import requests
3
4 # Базовый URL эндпойнта OpenAI, откуда запрашиваем список моделей
5 url = "https://api.openai.com/v1/models"
6
7 # Заголовки HTTP-запроса:
8 headers = {
9     # Токен авторизации в формате Bearer; здесь должен быть ваш реальный ключ
10     "Authorization": "Bearer sk-abc123...",
11     # Сообщаем серверу, что отправляем/принимаем JSON
12     "Content-Type": "application/json"
13 }
14
15 # Выполняем GET-запрос по URL с указанными заголовками
16 response = requests.get(url, headers=headers)
17
18 # Выводим HTTP-код ответа (200 — успех, 401 — ошибка авторизации и т. д.)
19 print(response.status_code)
20
21 # Выводим тело ответа, преобразованное из JSON в питоновский словарь
22 print(response.json())
```

OAuth2 — это протокол авторизации, который используется, когда нужно предоставить доступ от имени пользователя (например, доступ к данным GitHub или Google).

Обычно используется в два шага:

1. Получить **authorization code** (через браузер).
2. Обменять его на **access token** и использовать в API-запросах.

Пример из жизни: Вы логинитесь через GitHub в стороннем приложении — это и есть OAuth2.

Для LLM и агентных фреймворков чаще всего достаточно **API-ключей**, и OAuth2 знать нужно только для более сложных интеграций.

Никогда не храните ключи прямо в коде.

🎓 Резюме

- API-ключ — это ваш "пароль" для доступа к сервису.
- Он передаётся через заголовок `Authorization: Bearer ...`.
- Никогда не оставляйте ключи в коде.

Машинное обучение: основы

Машинное обучение (ML) — это область искусственного интеллекта, занимающаяся разработкой алгоритмов, которые автоматически извлекают закономерности из данных и используют их для прогнозирования или принятия решений. Его ключевые компоненты включают данные, модели и процесс обучения, где качество данных напрямую влияет на эффективность алгоритмов

Ключевые понятия

- **Объект (пример, sample)** — отдельная сущность, для которой требуется сделать прогноз или принять решение. Например, это может быть изображение, текст, пользователь или транзакция.
- **Признаки (features)** — измеримые характеристики объекта, используемые для описания его свойств. Например, для изображения это может быть цвет каждого пикселя, для пользователя — возраст и пол.
- **Признаковое пространство** — многомерное пространство, в котором каждый объект представлен вектором признаков.
- **Выборка (dataset)** — совокупность объектов, для которых известны значения признаков, а иногда и правильные ответы (разметка).
- **Модель** — формализованное представление зависимости между признаками объекта и целевым ответом. Модель может быть линейной, деревом решений, нейронной сетью и т.д.
- **Параметры модели** — настраиваемые величины внутри модели, которые определяются в процессе обучения для достижения наилучших результатов.
- **Обучение (training)** — процесс подбора параметров модели на основе обучающей выборки, чтобы минимизировать ошибку между предсказаниями модели и реальными ответами.
- **Функция потерь (loss function)** — количественная мера ошибки модели, которую необходимо минимизировать в процессе обучения.
- **Обобщающая способность (generalization)** — способность модели давать корректные прогнозы на новых, ранее не встречавшихся данных.
- **Переобучение (overfitting)** — ситуация, когда модель слишком точно запоминает обучающую выборку, теряя способность обобщать на новые данные

Основные типы обучения

Обучение с учителем (Supervised Learning)

Использует размеченные данные, где каждому примеру соответствует метка, которая уже есть. Примеры задач:

- **Классификация:** Определение категории объекта (например, спам/не спам).

Классификация относит объекты к дискретным категориям. Например, распознавание рукописных цифр или диагностика заболеваний по медицинским снимкам. Методы включают логистическую регрессию и деревья решений, где модель обучается на примерах с известными метками.

- **Регрессия:** Прогнозирование числовых значений (например, цены дома).

Регрессия предсказывает непрерывные числовые значения. Например, модель может оценивать пожизненную ценность клиента на основе возраста, истории покупок и демографических данных. Линейная регрессия, описываемая уравнением $y = ax + b$, минимизирует разницу между предсказанными и реальными значениями через остатки.

Обучение без учителя (Unsupervised Learning)

Работает с неразмеченными данными, выявляя скрытые структуры. Основной метод:

- **Кластеризация:** Группировка объектов по схожести (например, сегментация клиентов)

В кластеризации данные группируются без предварительных меток. Алгоритмы вроде K-средних находят естественные скопления объектов, что полезно для анализа рынка или обнаружения аномалий. Например, группировка клиентов по покупательскому поведению помогает развивать маркетинговые стратегии.

Представление данных

В современном анализе данных и машинном обучении огромное значение имеет корректное представление и обработка данных. Основные задачи — это эффективное хранение, быстрый доступ, преобразование и подготовка данных к анализу и моделированию. Поэтому мы рассмотрим ключевые структуры для представления данных в Python: `numpy.ndarray` и `pandas.DataFrame`, а также разберём различия между категориальными и числовыми признаками и методы кодирования категориальных признаков, в частности, one-hot-encoding.

Что такое `numpy.ndarray`

- `numpy.ndarray` — основной объект библиотеки NumPy для хранения и обработки многомерных массивов данных.
- Обеспечивает быстрые математические операции и эффективное хранение числовой информации.

Создание массивов

```
1 import numpy as np
2 arr = np.array([1, 2, 3, 4, 5]) # 1-D массив
3 arr2d = np.array([[1, 2, 3], [4, 5, 6]]) # 2-D массив
```

Базовые операции

NumPy поддерживает векторизованные операции:

```
1 data = np.array([1, 2])
2 ones = np.ones(2, dtype=int)
3 print(data + ones) # array([2, 3])
4 print(data - ones) # array([0, 1])
5 print(data * data) # array([1, 4])
6 print(data / data) # array([1., 1.])
```

Агрегирующие функции:

```
1 a = np.array([1, 2, 3, 4])
2 print(a.sum()) # 10
```

Для двумерных массивов:

```
1 b = np.array([[1, 1], [2, 2]])
2 print(b.sum(axis=0)) # array([3, 3]) — сумма по строкам (для каждого столбца)
3 print(b.sum(axis=1)) # array([2, 4]) — сумма по столбцам (для каждой строки)
```

`pandas.DataFrame`: базовые операции и индексация

Что такое `pandas.DataFrame`

- `pandas.DataFrame` — двумерная таблица с именованными строками и столбцами.
- Аналог таблицы в базе данных или листа Excel.
- Поддерживает разнородные по типу данные и мощные средства анализа.

Создание и чтение данных

```
1 import pandas as pd
2 df = pd.DataFrame({'Name': ['Alice', 'Bob'], 'Age': [25, 30]})
```

Базовые операции

- **Добавление/удаление столбцов:**

```
df['NewCol'] = ... ,
df.drop('Col', axis=1)
```

- **Фильтрация:**

```
df[df['Age'] > 25]
```

- **Сортировка:**

```
df.sort_values('Age')
```

- **Группировка и агрегация:**

```
df.groupby('Col').sum()
```

- **Обработка пропусков:**

```
df.fillna(0) ,
df.dropna()
```

- **Математические операции:**

```
df.add() ,
df.subtract() ,
df.multiply() ,
df.divide()
```

Индексация

- DataFrame имеет индекс — метки для строк (числовые, строковые и др.).
- Индекс используется для доступа к строкам и выравнивания данных.

```
1 df = pd.DataFrame({'Name': ['Alice', 'Bob'], 'Age': [25, 30]}, index=[10, 20])
2 print(df.index) # Index([10, 20], dtype='int64')
3
4 df.index = [100, 200] # изменение индекса
```

Доступ к строкам и столбцам:

```
1 df.loc[100] # строка с индексом 100
2 df.iloc[0] # первая строка
```

Категориальные vs. числовые признаки

Числовые признаки

- **Числовые (numeric) признаки** — переменные, принимающие количественные значения: целые или вещественные числа (например, возраст, доход, количество покупок).
- Используются напрямую в математических операциях и моделях.

Категориальные признаки

- **Категориальные (categorical) признаки** — переменные, принимающие значения из ограниченного множества категорий или классов (например, пол, цвет, страна).

- Бывают:
 - **Номинальные** (без порядка): цвет (красный, синий, зелёный)
 - **Порядковые** (с порядком): уровень образования (бакалавр, магистр, доктор)
- Требуют преобразования в числовую форму для большинства алгоритмов машинного обучения.

One-hot-encoding: преобразование категориальных признаков

Что такое one-hot-encoding

- Способ преобразования категориальных признаков в числовой формат.
- Для каждой категории создаётся отдельный бинарный столбец.
- Если объект принадлежит категории, соответствующий столбец получает значение 1, иначе — 0.

Пример

Цвет	Цвет_красный	Цвет_синий	Цвет_зелёный
Красный	1	0	0
Синий	0	1	0
Зеленый	0	0	1

Преимущества one-hot-encoding

- Не вводит ложного порядка между категориями.
- Удобен для большинства алгоритмов машинного обучения.

Недостатки

- При большом количестве уникальных категорий число новых признаков резко увеличивается (проклятие размерности).
- В некоторых моделях можно использовать альтернативные методы кодирования.

Реализация в pandas и sklearn

В pandas:

```
1 import pandas as pd
2 df = pd.DataFrame({'Color': ['red', 'blue', 'green']})
3 df_encoded = pd.get_dummies(df, columns=['Color'])
```

В scikit-learn:

```
1 from sklearn.preprocessing import OneHotEncoder
2 encoder = OneHotEncoder(sparse=False)
3 encoded = encoder.fit_transform(df[['Color']])
```

Сравнение категориальных и числовых признаков

Характеристика	Категориальные признаки	Числовые признаки
Значения	Ограниченное множество категорий	Числа (целые/вещественные)
Примеры	Пол, цвет, страна	Возраст, доход, рост
Требует кодирования	Да (one-hot, label encoding)	Нет, используется напрямую
Использование в моделях	Требует преобразования	Используется напрямую

Векторные представления текста и препроцессинг текстовых данных

Векторные представления текста позволяют преобразовать текстовые данные в числовой формат, пригодный для машинного обучения и анализа. Это фундаментальный этап в обработке естественного языка (NLP), так как большинство алгоритмов работают только с числовыми данными.

Токенизация — это процесс разбиения текста на отдельные элементы (токены): слова, предложения или даже символы. Этот шаг необходим для дальнейшей обработки текста и построения векторных представлений.

Виды токенизации:

Whitespace Tokenizer

Токенизация по пробелам — самый простой способ, когда текст разбивается по пробелам, табуляциям и переводам строк.

```
1 from nltk.tokenize import WhitespaceTokenizer
2
3 text = "This is an example text."
4 tokenizer = WhitespaceTokenizer()
5 tokens = tokenizer.tokenize(text)
6 print(tokens)
7 # ['This', 'is', 'an', 'example', 'text.']
```

Этот метод быстрый, но не учитывает знаки препинания и сложные случаи разметки.

Regex Tokenizer

Токенизация с помощью регулярных выражений позволяет более гибко управлять разбиением текста, например, исключать знаки препинания или выделять только слова определённого алфавита.

```
1 from nltk.tokenize import RegexpTokenizer
2
3 text = "Это пример: текста, с пунктуацией!"
4 tokenizer = RegexpTokenizer(r'\w+')
5 tokens = tokenizer.tokenize(text)
6 print(tokens)
7 # ['Это', 'пример', 'текста', 'с', 'пунктуацией']
```

Для выделения только русских слов:

```
1 import re
2
3 text = "one, два, три, four"
4 tokens = re.findall(r'[А-Яа-яёЁ]+', text)
5 print(tokens)
6 # ['два', 'три']
```

Этот способ позволяет гибко фильтровать токены по нужным критериям.

NLTK Tokenizers

NLTK предоставляет несколько токенизаторов:

- `word_tokenize` — разбивает текст на слова, учитывая пунктуацию.
- `TreebankWordTokenizer` — использует правила корпуса Penn Treebank.
- `RegexpTokenizer` — позволяет задать собственное регулярное выражение.

- `WhitespaceTokenizer` — разбивает по пробелам.

```
1 from nltk.tokenize import word_tokenize
2
3 text = "NLTK упрощает обработку текста."
4 tokens = word_tokenize(text)
5 print(tokens)
6 # ['NLTK', 'упрощает', 'обработку', 'текста', '.']
```

Bag-of-Words (BoW)

Модель «мешка слов» (Bag-of-Words) — это простейший способ векторизации текста. Каждый документ представляется вектором, где каждая позиция соответствует слову из словаря, а значение — количеству его вхождений.

```
1 from sklearn.feature_extraction.text import CountVectorizer
2
3 texts = ["Я люблю машинное обучение", "Машинное обучение – это интересно"]
4 vectorizer = CountVectorizer()
5 X = vectorizer.fit_transform(texts)
6 print(vectorizer.get_feature_names_out())
7 # ['интересно' 'люблю' 'машинное' 'обучение' 'это']
8 print(X.toarray())
9 # [[0 1 1 1 0]
10 #   [1 0 1 1 1]]
```

TF-IDF

TF-IDF (Term Frequency-Inverse Document Frequency) — более продвинутая модель, которая учитывает не только частоту слова в документе, но и его уникальность для всей коллекции документов. Это снижает вес часто встречающихся, но малозначимых слов (например, союзов и предлогов).

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2
3 texts = ["Я люблю машинное обучение", "Машинное обучение – это интересно"]
4 vectorizer = TfidfVectorizer()
5 X = vectorizer.fit_transform(texts)
6 print(vectorizer.get_feature_names_out())
7 # ['интересно' 'люблю' 'машинное' 'обучение' 'это']
8 print(X.toarray())
9 # Пример вывода:
10 # [[0.          0.7071  0.5       0.5       0.        ]
11 #   [0.5773     0.         0.4082  0.4082  0.5773]]
```

N-граммы

N-граммы — это последовательности из N слов, встречающихся подряд в тексте. Использование биграмм (2-грамм), триграмм (3-грамм) и более длинных последовательностей позволяет учитывать контекст и устойчивые выражения.

```
1 from sklearn.feature_extraction.text import CountVectorizer
2
3 texts = ["машинное обучение это интересно"]
4 vectorizer = CountVectorizer(ngram_range=(2, 2))
5 X = vectorizer.fit_transform(texts)
6 print(vectorizer.get_feature_names_out())
7 # ['машинное обучение' 'обучение это' 'это интересно']
```

```
8 print(X.toarray())
9 # [[1 1 1]]
```

Для выделения биграмм с помощью Gensim:

```
1 from gensim.models.phrases import Phrases, Phraser
2
3 sentences = [["машинное", "обучение", "это", "интересно"]]
4 bigram = Phrases(sentences, min_count=1, threshold=1)
5 phrases = Phraser(bigram)
6 for sent in sentences:
7     print(phrases[sent])
8 # ['машинное_обучение', 'это', 'интересно']
```

N-граммы позволяют выявлять устойчивые выражения и улучшать качество векторизации³.

Фильтрация стоп-слов

Стоп-слова — это часто встречающиеся, но малозначимые слова (например, "и", "в", "на"), которые обычно удаляют перед анализом текста.

```
1 from nltk.corpus import stopwords
2 from nltk.tokenize import word_tokenize
3
4 import nltk
5 nltk.download('stopwords')
6 nltk.download('punkt')
7
8 text = "NLTK помогает в удалении стоп-слов из текста."
9 tokens = word_tokenize(text)
10 stop_words = set(stopwords.words('russian'))
11 filtered_tokens = [word for word in tokens if word.lower() not in stop_words]
12 print(filtered_tokens)
13 # ['NLTK', 'помогает', 'удалении', 'стоп-слов', 'текста', '.']
```

В `CountVectorizer` и `TfidfVectorizer` можно указать список стоп-слов:

```
1 from sklearn.feature_extraction.text import CountVectorizer
2
3 texts = ["Это пример текста для анализа"]
4 vectorizer = CountVectorizer(stop_words=['это', 'для'])
5 X = vectorizer.fit_transform(texts)
6 print(vectorizer.get_feature_names_out())
7 # ['анализа' 'пример' 'текста']
```

Фильтрация стоп-слов повышает качество анализа, убирая шум из текстовых данных

Основные модели машинного обучения

Линейная регрессия

Основная идея:

Линейная регрессия ищет линейную зависимость между входными признаками x и целевой переменной y :

$$y = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$$

Обучение:

Коэффициенты w подбираются так, чтобы минимизировать среднеквадратичную ошибку (MSE) между предсказанными и реальными значениями:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Основные параметры:

- Регуляризация (L1/L2): штраф за большие веса, помогает бороться с переобучением

L1 регуляризация (Lasso)

- Добавляет к функции потерь сумму абсолютных значений весов:

$$L = L_0 + \lambda \sum_{i=1}^n |w_i|$$

- Способна обнулять веса незначимых признаков, что приводит к разреженным решениям и фактически осуществляет отбор признаков.
- Может быть менее эффективна с градиентным спуском из-за недифференцируемости в нуле, но популярна для отбора признаков.

L2 регуляризация (Ridge)

- Добавляет к функции потерь сумму квадратов весов:

$$L = L_0 + \lambda \sum_{i=1}^n w_i^2$$

- Уменьшает все веса, но не обнуляет их полностью, делая модель более "гладкой" и устойчивой к шуму.
- Хорошо работает с градиентным спуском и полезна при большом количестве коррелированных признаков

Переобучение и недообучение:

- Переобучение — при слишком сложной модели (много признаков, отсутствие регуляризации)
- Недообучение — если модель слишком простая или признаки неинформативны

Пример на Python:

```
1 # Импортируем необходимые классы:
2 # LinearRegression — классический метод ОЛС-регрессии без регуляризации
3 # Ridge — линейная регрессия с L2-регуляризацией (гребневой)
4 from sklearn.linear_model import LinearRegression, Ridge
5
6 # Функции для разбиения данных и оценки качества
7 from sklearn.model_selection import train_test_split
8 from sklearn.metrics import mean_squared_error
9
10 # Генератор искусственного набора данных для регрессии
11 from sklearn.datasets import make_regression
12
13 # 1. Генерируем выборку из 100 объектов, 2 признаков и добавляем шум (noise=10)
```



```

14 X, y = make_regression(
15     n_samples=100,
16     n_features=2,
17     noise=10,
18     random_state=42 # фиксируем сид, чтобы пример был воспроизводимым
19 )
20
21 # 2. Делим данные на обучающую и тестовую части (80 % / 20 %)
22 X_train, X_test, y_train, y_test = train_test_split(
23     X, y,
24     test_size=0.2,
25     random_state=42 # фиксируем разбиение
26 )
27
28 # 3. Создаём и обучаем модель без регуляризации
29 lr = LinearRegression()
30 lr.fit(X_train, y_train)
31
32 # 4. Делаем прогноз и считаем среднеквадратичную ошибку (MSE)
33 y_pred = lr.predict(X_test)
34 print("MSE (без регуляризации):", mean_squared_error(y_test, y_pred))
35
36 # 5. То же самое, но с L2-регуляризацией (гиперпараметр alpha=1.0)
37 ridge = Ridge(alpha=1.0)
38 ridge.fit(X_train, y_train)
39 y_pred_ridge = ridge.predict(X_test)
40 print("MSE (с L2-регуляризацией):", mean_squared_error(y_test, y_pred_ridge))

```

Логистическая регрессия

Основная идея:

Логистическая регрессия применяется для задач классификации (например, 0 или 1):

$$P(y = 1 | x) = \frac{1}{1 + \exp(-(w_0 + w_1x_1 + \dots + w_nx_n))}$$

Обучение:

Коэффициенты подбираются так, чтобы минимизировать функцию потерь — кросс-энтропию:

$$L = -\frac{1}{N} \sum_{i=1}^N [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)]$$

Основные параметры:

- Регуляризация (L1/L2): снижает переобучение
- Порог классификации: влияет на чувствительность/специфичность

Переобучение и недообучение:

- Переобучение — при большом количестве признаков и отсутствии регуляризации
- Недообучение — если модель не способна уловить зависимости (например, данные не линейно разделимы)

Пример на Python:

```

1 # Импортируем модель логистической регрессии
2 from sklearn.linear_model import LogisticRegression
3 # Генератор искусственного датасета для классификации
4 from sklearn.datasets import make_classification
5 # Разделение данных на обучающую и тестовую выборки
6 from sklearn.model_selection import train_test_split

```

```

7 # Метрика для оценки точности модели
8 from sklearn.metrics import accuracy_score
9
10 # Создаём синтетический набор данных:
11 # - 100 образцов
12 # - 4 признака, из которых 2 информативных и 0 лишних
13 # - фиксируем random_state для воспроизводимости
14 X, y = make_classification(
15     n_samples=100,
16     n_features=4,
17     n_informative=2,
18     n_redundant=0,
19     random_state=42
20 )
21
22 # Делим данные: 80 % — обучение, 20 % — тест
23 X_train, X_test, y_train, y_test = train_test_split(
24     X, y,
25     test_size=0.2,
26     random_state=42
27 )
28
29 # Создаём объект логистической регрессии:
30 # - penalty='l2' — L2-регуляризация (по умолчанию)
31 # - C=1.0 — коэффициент регуляризации (меньше — сильнее регуляризация)
32 logreg = LogisticRegression(penalty='l2', C=1.0)
33
34 # Обучаем модель на обучающей выборке
35 logreg.fit(X_train, y_train)
36
37 # Делаем прогноз для тестовой выборки
38 y_pred = logreg.predict(X_test)
39
40 # Выводим точность модели (доля правильных ответов)
41 print("Accuracy:", accuracy_score(y_test, y_pred))

```

Метод k-ближайших соседей (kNN)

Основная идея:

Для нового объекта находятся k ближайших (по метрике) объектов из обучающей выборки. Класс определяется большинством голосов среди этих соседей.

Обучение:

Модель не требует явного этапа обучения — все вычисления происходят на этапе предсказания.

Основные параметры:

- k (число соседей): главный параметр
- Метрика расстояния: Евклидова, Манхэттенская и др.
- Взвешивание соседей: ближние могут иметь больший вес

Переобучение и недообучение:

- Переобучение — при малых k (например, $k = 1$), модель чувствительна к шуму
- Недообучение — при больших k , модель становится слишком грубой

Пример на Python:

```

1 # Импортируем необходимые компоненты из scikit-learn
2 from sklearn.neighbors import KNeighborsClassifier # алгоритм k-ближайших соседей (k-NN)
3 from sklearn.datasets import load_iris          # встроенный датасет «Ирисы Фишера»
4 from sklearn.model_selection import train_test_split # функция для разбиения данных на train/test

```

```

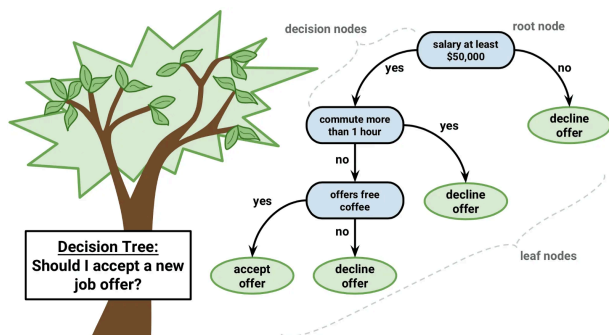
5 from sklearn.metrics import accuracy_score # метрика точности классификации
6
7 # Загружаем признаки X и метки классов y
8 X, y = load_iris(return_X_y=True) # X — числовые характеристики цветов, y — вид ириса
9
10 # Делим выборку: 80 % уйдёт на обучение, 20 % — на проверку
11 X_train, X_test, y_train, y_test = train_test_split(
12     X, y,
13     test_size=0.2, # доля тестовой части
14     random_state=42 # фиксируем сид для воспроизводимости
15 )
16
17 # Создаём экземпляр классификатора k-NN с тремя соседями
18 knn = KNeighborsClassifier(n_neighbors=3) # k = 3
19
20 # «Обучаем» модель (k-NN просто запоминает обучающие точки)
21 knn.fit(X_train, y_train)
22
23 # Делаем предсказания для тестовой выборки
24 y_pred = knn.predict(X_test)
25
26 # Вычисляем и выводим точность модели
27 print("Accuracy (k=3):", accuracy_score(y_test, y_pred))

```

Дерево решений

Основная идея:

Дерево решений — древовидная структура, где в каждом узле происходит разбиение данных по признаку.



Обучение:

Дерево строится жадно: на каждом шаге выбирается признак и порог, дающие наилучшее разбиение по выбранному критерию (например, прирост информации, индекс Джини).

Индекс Джини (Gini impurity) — это метрика, используемая при обучении дерева решений для оценки "чистоты" узлов, то есть однородности классов в каждом подмножестве после разбиения. Он показывает вероятность того, что случайно выбранный элемент будет ошибочно классифицирован, если его метка определяется случайно в соответствии с распределением классов в данном узле.

Формула индекса Джини:

$$\text{Gini}(Q) = 1 - \sum_{i=1}^n p_i^2$$

где p_i — доля объектов класса i в рассматриваемом множестве, n — число классов.

- Значение индекса Джини изменяется от 0 (все объекты в узле принадлежат одному классу — "чистый" узел) до максимума (классы распределены равномерно).
- При построении дерева решений алгоритм выбирает такие разбиения, которые минимизируют индекс Джини, то есть делают дочерние узлы максимально однородными по составу классов.
- Индекс Джини широко используется в алгоритме CART для задач классификации.

Таким образом, минимизация индекса Джини на каждом шаге обучения дерева решений позволяет строить более точные и интерпретируемые модели классификации

Основные параметры:

- Глубина дерева: максимальное количество уровней
- Минимальное число объектов в листе: предотвращает слишком мелкие разбиения
- Критерий разбиения: Джини, энтропия и др.

Переобучение и недообучение:

- Переобучение — глубокие деревья полностью "запоминают" обучающую выборку
- Недообучение — слишком мелкие деревья не способны уловить структуру данных

Пример на Python:

```
1 # Импортируем необходимые классы и функции из scikit-learn
2 from sklearn.tree import DecisionTreeClassifier # алгоритм дерева решений
3 from sklearn.datasets import load_wine        # встроенный датасет «Вино»
4 from sklearn.model_selection import train_test_split # разбиение на обучающую/тестовую выборки
5 from sklearn.metrics import accuracy_score      # метрическая функция для оценки точности
6
7 # Загружаем набор данных: X — матрица признаков, y — вектор целевых меток (классов вин)
8 X, y = load_wine(return_X_y=True)
9
10 # Делим данные: 80 % — обучение, 20 % — тест.
11 # random_state фиксируем, чтобы эксперимент был воспроизводимым.
12 X_train, X_test, y_train, y_test = train_test_split(
13     X, y, test_size=0.2, random_state=42
14 )
15
16 # Создаём модель дерева решений.
17 # max_depth=3 ограничивает глубину, предотвращая переобучение,
18 # random_state фиксирует генератор случайностей внутри алгоритма.
19 tree = DecisionTreeClassifier(max_depth=3, random_state=42)
20
21 # Обучаем модель на обучающей выборке.
22 tree.fit(X_train, y_train)
23
24 # Получаем предсказания классов для тестовой выборки.
25 y_pred = tree.predict(X_test)
26
27 # Считаем долю правильных ответов (accuracy) и выводим её.
28 print("Accuracy (max_depth=3):", accuracy_score(y_test, y_pred))
29
```

Случайный лес

Основная идея:

Случайный лес — ансамблевый метод, объединяющий множество деревьев решений для повышения точности и устойчивости.

Обучение:

- Строится множество деревьев (обычно десятки-сотни)
- Для каждого дерева случайно выбирается подмножество признаков для разбиений

Основные параметры:

- Число деревьев: больше — устойчивее, но дольше обучение

- Максимальная глубина дерева: влияет на сложность каждого дерева
- Размер подвыборки признаков: уменьшает корреляцию между деревьями

Переобучение и недообучение:

- Переобучение — обычно менее выражено, чем у одиночных деревьев, но возможно при слишком глубоких деревьях
- Недообучение — если деревья слишком мелкие или их мало

Пример на Python:

```
1 # --- Импорт необходимых библиотек ---
2 from sklearn.ensemble import RandomForestClassifier # Модель «Случайный лес»
3 from sklearn.datasets import load_breast_cancer   # Готовый набор данных «Breast Cancer»
4 from sklearn.model_selection import train_test_split # Разделение выборки на обучение/тест
5 from sklearn.metrics import accuracy_score        # Метрика точности классификации
6
7 # --- Загрузка данных ---
8 # X – матрица признаков (n_samples × n_features)
9 # y – вектор ответов (0 = злокачественная, 1 = доброкачественная опухоль)
10 X, y = load_breast_cancer(return_X_y=True)
11
12 # --- Деление на обучающую и тестовую выборки ---
13 # 20 % данных пойдут в тест, random_state фиксируем для воспроизводимости
14 X_train, X_test, y_train, y_test = train_test_split(
15     X, y,
16     test_size=0.20,
17     random_state=42
18 )
19
20 # --- Создание и обучение модели ---
21 # n_estimators=100 → 100 деревьев
22 # max_depth=5 → ограничиваем глубину, чтобы избежать переобучения
23 # random_state=42 → фиксируем случайность
24 rf = RandomForestClassifier(
25     n_estimators=100,
26     max_depth=5,
27     random_state=42
28 )
29 rf.fit(X_train, y_train) # «Учимся» на обучающих данных
30
31 # --- Предсказания на тестовой выборке ---
32 y_pred = rf.predict(X_test)
33
34 # --- Оценка качества ---
35 # accuracy_score вычисляет долю правильных предсказаний
36 print("Accuracy (Random Forest):", accuracy_score(y_test, y_pred))
```

Влияние параметров на переобучение и недообучение

Модель	Основные параметры	Влияние на переобучение/недообучение
Линейная регрессия	Регуляризация, признаки	Без регуляризации — переобучение; сильная регуляризация — недообучение
Логистическая регрессия	Регуляризация, признаки	Аналогично линейной регрессии

kNN	k, метрика	Малый k — переобучение; большой k — недообучение
Дерево решений	Глубина, мин. объектов в листе	Глубокое дерево — переобучение; мелкое — недообучение
Случайный лес	Число деревьев, глубина	Мало деревьев/мелкие — недообучение; слишком глубокие деревья — переобучение

Оценка качества модели

Оценка качества модели — ключевой этап в процессе машинного обучения, позволяющий объективно определить, насколько хорошо модель решает поставленную задачу. Правильная оценка помогает выбрать оптимальную модель, избежать переобучения и повысить надежность решений на новых данных.

Тренировочное, валидационное, тестовое разбиение

Для корректной оценки качества модели принято делить исходные данные на несколько частей:

- **Тренировочная выборка (train)** — используется для обучения модели.
- **Валидационная выборка (validation)** — применяется для подбора гиперпараметров и контроля переобучения.
- **Тестовая выборка (test)** — предназначена для финальной оценки качества модели на ранее не виденных данных.

Один из самых популярных способов разделения — функция `train_test_split` из библиотеки `scikit-learn`:

```
1 from sklearn.model_selection import train_test_split
2
3 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Кросс-валидация (K-fold)

Кросс-валидация — более надежный способ оценки, при котором данные делятся на K частей (фолдов). Модель обучается K раз, каждый раз используя одну из частей как тестовую, а остальные — для обучения. Итоговая оценка — среднее по всем фолдам.

```
1 from sklearn.model_selection import cross_val_score
2 from sklearn.ensemble import RandomForestClassifier
3
4 scores = cross_val_score(RandomForestClassifier(), X, y, cv=5)
5 print(f"Средняя точность: {scores.mean():.2f}")
```

Метрики оценки качества

Выбор метрики зависит от типа задачи: классификация или регрессия.

Классификация

- **Accuracy (Точность)** — доля правильно классифицированных объектов среди всех объектов. Простая и интуитивная метрика, но может быть неинформативна при дисбалансе классов.

```
1 from sklearn.metrics import accuracy_score
2
3 accuracy = accuracy_score(y_test, y_pred)
4 print(f'Accuracy: {accuracy:.2f}')
```

- **Precision (Точность)** — доля истинно положительных среди всех, предсказанных как положительные. Важно, если цена ложноположительных высока
- **Recall (Полнота)** — доля истинно положительных среди всех реально положительных. Важно, если критичны пропуски положительных случаев.
- **F1-score** — гармоническое среднее между precision и recall. Используется при необходимости баланса между этими метриками.

▼

```
1 from sklearn.metrics import precision_score, recall_score, f1_score
2
3 precision = precision_score(y_test, y_pred)
4 recall = recall_score(y_test, y_pred)
5 f1 = f1_score(y_test, y_pred)
6 print(f'Precision: {precision:.2f}, Recall: {recall:.2f}, F1-score: {f1:.2f}')
```

- **ROC-AUC** — площадь под ROC-кривой. Оценивает способность модели различать классы при разных порогах.

▼

```
1 from sklearn.metrics import roc_auc_score
2
3 roc_auc = roc_auc_score(y_test, y_pred_proba)
4 print(f'ROC-AUC: {roc_auc:.2f}')
```

Регрессия

MSE (Mean Squared Error, среднеквадратичная ошибка) — это одна из самых популярных метрик для оценки качества моделей в задачах регрессии. Она рассчитывается как среднее арифметическое квадратов разностей между фактическими значениями и предсказаниями модели:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

где y_i — истинные значения, $\hat{y}_i$ — предсказанные моделью значения, n — количество наблюдений.

Особенности MSE:

- Квадратичное возведение ошибки приводит к тому, что большие ошибки штрафуются сильнее, чем маленькие, поэтому MSE чувствительна к выбросам.
- MSE часто используется как функция потерь при обучении моделей, например, линейной регрессии и нейронных сетей.
- Значение MSE не имеет верхней границы и выражается в квадрате единиц измерения целевой переменной, что иногда затрудняет интерпретацию результата
- **RMSE (Root Mean Squared Error)** — корень из средней квадратичной ошибки. Чем меньше значение, тем лучше модель предсказывает числовые значения. Эту метрику используют чаще, так как она более информативна (выражается не в квадрате единиц, а в самих единицах)

▼

```
1 from sklearn.metrics import mean_squared_error
2 import numpy as np
3
4 rmse = np.sqrt(mean_squared_error(y_test, y_pred))
5 print(f'RMSE: {rmse:.2f}')
```

Сравнение основных метрик

Метрика	Описание	Применение
Accuracy	Доля верных предсказаний	Классификация
Precision	Доля истинно положительных среди предсказанных положительных	Классификация (важна цена FP)
Recall	Доля истинно положительных среди всех положительных	Классификация (важна цена FN)
F1-score	Баланс между precision и recall	Классификация (дисбаланс)
ROC-AUC	Площадь под ROC-кривой	Классификация (оценка порогов)
RMSE	Корень из средней квадратичной ошибки	Регрессия

Введение в эмбединги и семантические векторы

Эмбединги — это способ представления слов, фраз или целых предложений в виде плотных векторов фиксированной размерности, отражающих их смысловое содержание. Такие векторы позволяют алгоритмам машинного обучения работать с текстом как с числовыми данными, сохраняя при этом семантическую информацию.

Классические эмбединги: Word2Vec и GloVe

Word2Vec

Word2Vec — это модель, которая преобразует каждое слово в вектор фиксированной длины, основываясь на его окружении в тексте. Существуют две основные архитектуры:

- **CBOW (Continuous Bag of Words):** предсказывает текущее слово по окружающим его словам.
- **Skip-gram:** наоборот, предсказывает окружающие слова по текущему.

Word2Vec строит эмбединги на основе локального контекста — то есть, учитывает только слова, расположенные рядом с целевым словом в предложении. Это ограничивает модель, так как она не использует глобальную статистику по всему корпусу.

Этапы работы:

- Чтение корпуса и построение словаря.
- Преобразование слов в векторы (one-hot).
- Обучение нейронной сети на задаче предсказания слова по контексту (CBOW) или контекста по слову (Skip-gram).
- Получение итоговой матрицы эмбедингов, где каждая строка — вектор слова.

Применение

- Кластеризация и классификация текстов.
- Поиск синонимов и аналогий (word analogies).
- Улучшение качества других моделей обработки естественного языка (NLP).
- Слова, часто встречающиеся в похожих контекстах, получают близкие вектора.
- Можно выполнять операции: $\text{вектор(король)} - \text{вектор(мужчина)} + \text{вектор(женщина)} \approx \text{вектор(королева)}$.

GloVe (Global Vectors for Word Representation)

GloVe — это модель, которая строит эмбединги, используя матрицу совместной встречаемости слов во всём корпусе. Она минимизирует разницу между скалярным произведением векторов слов и логарифмом вероятности их совместного появления. Таким образом, GloVe сочетает локальную и глобальную статистику, что позволяет лучше захватывать семантические связи между словами.

Основная идея и принцип работы

- **Глобальная статистика:** В отличие от Word2Vec, который строит эмбединги на основе локального контекста (окна слов), GloVe использует *глобальную статистику совместной встречаемости слов* во всём корпусе текстов.
- **Матрица совместной встречаемости:** Алгоритм строит большую матрицу, где каждая ячейка отражает, сколько раз пара слов встречается в одном контексте (например, в пределах определённого окна).
- **Факторизация матрицы:** GloVe обучается на этой матрице с помощью разложения (факторизации), чтобы получить такие векторы слов, произведение которых аппроксимирует логарифм вероятности совместного появления пары слов.
- **Семантические отношения:** Векторы, полученные с помощью GloVe, отражают семантические и синтаксические связи между словами. Например, вектора слов «король» и «королева» будут ближе друг к другу, чем к вектору слова «яблоко».

Применение

- **Готовые эмбединги:** Существуют заранее обученные модели GloVe на больших корпусах (например, Wikipedia, Common Crawl), которые можно использовать для различных NLP-задач — классификации текстов, поиска, кластеризации, анализа тональности и др.
- **Векторизация текста:** После загрузки эмбедингов каждое слово преобразуется в фиксированный вектор, а для работы с предложениями или документами их векторы обычно усредняются или агрегируются другим способом

Сравнение Word2Vec и GloVe:

Характеристика	Word2Vec	GloVe
Основная идея	Локальный контекст (окна слов)	Глобальная статистика (матрица совместности)
Архитектура	Нейронная сеть (CBOW, Skip-gram)	Факторизация матрицы
Контекстуальность	Нет (статичные векторы)	Нет (статичные векторы)
Примеры использования	Классификация, анализ тональности	Классификация, анализ тональности

Контекстуальные эмбединги: Sentence-BERT и sentence-embeddings

Традиционные эмбединги (Word2Vec, GloVe) выдают для каждого слова один и тот же вектор, независимо от контекста. Это приводит к тому, что слова с разными значениями (например, "bank" — берег или банк) будут иметь одинаковое представление.

Sentence-BERT и другие sentence-transformers решают эту проблему, создавая эмбединги для целых предложений или фраз, учитывая порядок слов, грамматику и общий смысл. Sentence-BERT основан на архитектуре BERT, но оптимизирован для получения устойчивых векторов предложений, которые можно сравнивать между собой по косинусному сходству. Эта модель была предложена Нильсом Раймерсом и Ириной Гуревич в 2019 году и быстро стала стандартом для задач, где требуется сравнивать смысловые значения предложений.

Как работает Sentence-BERT

- **Модификация BERT:**
В стандартном BERT для вычисления схожести между двумя предложениями каждую пару предложений нужно подавать вместе, что делает процесс крайне ресурсоёмким для задач ранжирования или поиска по большому количеству текстов. Sentence-BERT решает эту проблему с помощью би-энкодерной архитектуры: каждое предложение независимо пропускается через модель, и на выходе получается вектор фиксированной размерности.
- **Эмбединги предложений:**
Каждое предложение преобразуется в плотный вектор (например, размерностью 384 или 768), который отражает его смысл. Близкие по смыслу предложения будут иметь близкие векторы в этом пространстве.
- **Сравнение предложений:**
Для оценки схожести между предложениями используется косинусное расстояние между их эмбедингами. Значение, близкое к 1, означает высокую смысловую близость, а к 0 — отсутствие связи.

Применение Sentence-BERT

- **Семантический поиск:**
Позволяет находить релевантные документы или предложения по смыслу, а не только по совпадению ключевых слов.
- **Кластеризация и группировка текстов:**
Схожие по смыслу предложения группируются вместе, что важно для анализа больших массивов данных.

- **Оценка релевантности и качества контента:**

Используется для аудита SEO-контента, оценки соответствия страниц поисковым запросам и улучшения структуры сайта.

- **Системы вопросов-ответов, чат-боты, анализ тональности:**

Благодаря глубокому пониманию смысла текста SBERT применяется в самых разных NLP-задачах.

Метрики сходства: косинусное сходство

Чтобы определить, насколько два вектора (например, эмбединги слов или предложений) похожи, часто используют **косинусное сходство**:

$$\text{cosine}_{\text{similarity}}(A, B) = \frac{A \cdot B}{|A| \cdot |B|}$$

- Значение 1 — векторы идентичны по направлению (максимальная схожесть).
- Значение 0 — векторы ортогональны (нет сходства).
- Значение -1 — векторы противоположны.

Поиск ближайших соседей

Для поиска наиболее похожих векторов (например, для задачи семантического поиска) используют алгоритмы поиска ближайших соседей. В Python популярны два инструмента:

- **sklearn.neighbors.NearestNeighbors** — классический метод поиска ближайших соседей в небольших и средних по размеру наборах данных.
- **FAISS** — библиотека от Facebook для быстрого поиска по большим коллекциям векторов, поддерживает GPU.

Примеры на Python

Word2Vec (gensim)

```
1 from gensim.models import Word2Vec
2
3 # Пример корпуса
4 sentences = [["машинное", "обучение", "это", "интересно"], ["обучение", "нейронных", "сетей"]]
5 model = Word2Vec(sentences, vector_size=100, window=5, min_count=1, workers=4)
6
7 # Вектор для слова "обучение"
8 vector = model.wv['обучение']
9 print(vector)
```

Sentence-BERT (sentence-transformers)

```
1 from sentence_transformers import SentenceTransformer, util
2
3 model = SentenceTransformer('all-MiniLM-L6-v2')
4 sentences = ["Как дела?", "Что нового?", "Как твои дела?"]
5
6 embeddings = model.encode(sentences)
7 similarity = util.cos_sim(embeddings[0], embeddings[2])
8 print(f"Cosine similarity: {similarity.item():.4f}")
```

Косинусное сходство

```
1 python
2 from sklearn.metrics.pairwise import cosine_similarity
3 import numpy as np
4
5 a = np.random.rand(1, 100)
6 b = np.random.rand(1, 100)
7 print(cosine_similarity(a, b))
```

Поиск ближайших соседей (sklearn и FAISS)

sklearn

```
1 from sklearn.neighbors import NearestNeighbors
2
3 # Пусть у нас есть массив эмбедингов
4 X = np.random.rand(100, 128)
5 nbrs = NearestNeighbors(n_neighbors=3, metric='cosine').fit(X)
6 distances, indices = nbrs.kneighbors([X[0]])
7 print(indices)
```

FAISS

```
1 import faiss
2 import numpy as np
3
4 d = 128 # размерность векторов
5 X = np.random.rand(1000, d).astype('float32')
6 index = faiss.IndexFlatL2(d)
7 index.add(X)
8
9 D, I = index.search(X[:5], 3) # 3 ближайших соседа для первых 5 векторов
10 print(I)
```

Ссылки на дополнительную литературу и лекции

- [Учебник по машинному обучению от ШАД](#)
- Hastie T., Tibshirani R, Friedman J. The Elements of Statistical Learning (2nd edition). Springer, 2009.
- Bishop C. M. Pattern Recognition and Machine Learning. Springer, 2006.
- Mohri M., Rostamizadeh A., Talwalkar A. Foundations of Machine Learning. MIT Press, 2012.
- Murphy K. Machine Learning: A Probabilistic Perspective. MIT Press, 2012.
- Mohammed J. Zaki, Wagner Meira Jr. Data Mining and Analysis. Fundamental Concepts and Algorithms. Cambridge University Press, 2014.
- Willi Richert, Luis Pedro Coelho. Building Machine Learning Systems with Python. Packt Publishing, 2013.

Курсы по машинному обучению и анализу данных

- [Курс по машинному обучению К.В. Воронцова](#)
- [Введение в машинное обучение \(онлайн-курс НИУ ВШЭ\)](#)